

Cel: Nabycie umiejętności korzystania ze środowiska programistycznego i konfiguratora układów peryferyjnych mikrokontrolera, programowania zestawu mikrokontrolerowego oraz podstawowej analizy działania programu (debugger) zrealizowanego z wykorzystaniem warstwy abstrakcyjnej (HAL - hardware abstraction layer).

Zadanie 1A [3 punkty]:

Korzystając z bibliotek STM32F3 HAL zrealizować program symulujący losowy rzut kostką zgodnie z poniższymi założeniami:

1. Po wciśnięciu przycisku *USER B1* (PA0) na module mikrokontrolerowym ma się aktywować pseudo-losowa liczba diod led podłączonych do pinów wyjściowych PE8-PE13 portu E. Stan ten ma być utrzymywany do momentu kolejnego wciśnięcia przycisku. Wyniki mają znajdować się w zakresie od 1 do 6.
2. Jeśli przycisk nie jest wciśnięty, to aktywność mikrokontrolera ma być sygnalizowana migającą diodą led LD6 podłączoną do pinu PE14. Częstotliwość zmiany stanu ma być równa wynikowi losowania * 0,5 (0,5Hz-3Hz), przed pierwszym losowaniem częstotliwość zmiany stanu ma wynosić 1Hz.
3. Wynik ma być uzyskiwany pseudo-losowo, można do tego celu wykorzystać zmieniającą się cyklicznie wartość zmiennej. W momencie wciśnięcia przycisku, na podstawie aktualnej wartości zmiennej, wyznaczany jest wynik losowania w zadanym powyżej zakresie.

Zadanie 1B [4 punkty]:

Korzystając z bibliotek STM32F3 HAL zrealizować program obsługujący cyfrowe wyprowadzenia wejściowe i wyjściowe mikrokontrolera zestawu STM32F3DISCOVERY, zgodnie z poniższymi założeniami:

1. Diody *LD3-LD10* wykorzystać jako wyświetlacz binarny, gdzie poszczególne diody reprezentują stan logiczny bitów według poniższej tabeli:

Bit	7	6	5	4	3	2	1	0
Dioda	LD6	LD8	LD10	LD9	LD7	LD5	LD3	LD4
Pin	PE15	PE14	PE13	PE12	PE11	PE10	PE9	PE8

2. Po zresetowaniu mikrokontrolera program prezentuje na wyświetlaczu binarnym kolejne liczby całkowite zakresu 0-63 zaczynając od wartości 0.
3. Zmiana stanu wyświetlacza binarnego na kolejną wartość następuje co około 500ms.

4. Przytrzymanie przycisku *USER B1* (PA0) powoduje naprzemienne zapalanie i gaszenie wszystkich diod wyświetlacza z częstotliwością około 4Hz.
5. Po wystąpieniu zbocza opadającego na wejściu PA0 (puszczenie przycisku *USER B1*) program zmienia kierunek odliczania (licząc od ostatnio uzyskanej wartości).
6. Przekroczenie zakresu 0-63 powoduje „przekroczenie” wyświetlanej wartości (przy liczeniu w górę 63->0, przy liczeniu w dół 0->63)

Wskazówki ogólne:

- Wyprowadzenia poszczególnych linii wejściowych (przycisk USER) oraz wyjściowych (diody) są przedstawione na ostatniej stronie schematu modułu STM32F3DISCOVERY (schemat udostępniony w materiałach na ePortalu)
- Szkielet programu i konfigurację poszczególnych układów peryferyjnych należy przygotować w konfiguratorze CubeMX. Dodając kod programu do wygenerowanych plików należy zwracać uwagę aby był on wpisywany w sekcjach pomiędzy komentarzami „USER CODE BEGIN” i „USER CODE END”. Wpisanie kodu w tych sekcjach nie spowoduje utraty napisanego programu przy rekonfiguracji w CubeMX.
- Deklarując zmienne należy korzystać ze standardowych typów danych języka C (np. `uint8_t` umożliwia deklarację zmiennej 8 bitowej, bez znaku, o zakresie wartości 0-255)
- Pomoc/komentarz dotyczący zasad wykorzystania danej funkcji oraz opisy typów zmiennych wyświetlane są automatycznie po najechaniu wskaźnikiem myszy na nazwę wpisanej funkcji/zmiennej.
- Rozliczenie zadania obejmuje także ocenę umiejętności posługiwania się **debuggerem**, w zakresie analizy krokowej wykonywania programu oraz poszukiwania błędów.

Wskazówki dotyczące obsługi wejść i wyjść:

- Bezpośredni dostęp do linii wyjściowych portu można uzyskać poprzez rejestr wyjściowy ***GPIOE->ODR*** (GPIOE jest wskaźnikiem do struktury reprezentującej port GPIOE, a operator `->` pozwala na dostęp do konkretnego rejestru tej struktury)
- Należy zwrócić uwagę na to, że diody wyświetlacza są podłączone do starszego bajtu 16-bitowego portu E (bity 8-15). Wyświetlaną liczbę można przenieść na starszy bajt portu za pomocą operatora przesunięcia bitowego (`<<`) lub poprzez mnożenie przez potęgę liczby 2.
- Aby wprowadzić opóźnienia synchroniczne (blokujące) można skorzystać z funkcji ***HAL_Delay()*** dostępnej w bibliotece HAL. Funkcja przyjmuje argument wyrażony w milisekundach (`uint32_t`). W mikrokontrolerach STM32 funkcja ta wykorzystuje licznik systemowy (SysTick) do obsługi czasu oczekiwania.
- Odczytanie stanu wybranego pinu portu można zrealizować z użyciem funkcji ***HAL_GPIO_ReadPin(GPIOx, GPIO_Pin)***, pierwszy argument funkcji jest wskaźnikiem do struktury wybranego portu (np. GPIOA), drugi argument to maska pinu (np. `GPIO_PIN_0` czyli `0x0001`). Analogicznie do zapisu stanu portu można wykorzystać funkcję ***HAL_GPIO_WritePin(GPIOx, GPIO_Pin, PinState)***.
- Zmianę stanu wybranego pinu portu można uzyskać bez zapamiętywania i negowania jego stanu za pomocą funkcji ***HAL_GPIO_TogglePin()***
- szczegółowy opis funkcji biblioteki HAL można znaleźć w dokumencie UM1786 Description of STM32F3 HAL and low-layer drivers, np. funkcja `HAL_GPIO_ReadPin` opisana jest w punkcie 23.2.5.